applied sciences

MDPI

Article

Resilient Software Design Through Cognitive-Aware Antipattern Propagation in 4+1 Architectural Views

Roberto Andrade [1,†,‡], Jenny Torres [2,‡], Iván Ortiz-Garcés [3,*,‡] and Jorge Segovia [2]

[1] Colegio Politecnico, Universidad San Francisco de Quito, Quito 170901, Ecuador; randradep@usfq.edu.ec
[2] Facultad de Ingenieria en Sistemas, Escuela Politecnica Nacional, Quito 170109, Ecuador; jenny.torres@epn.edu.ec (J.T.); jorge.segovia@epn.edu.ec (J.S.)
[3] Escuela de Ingeniería en Ciberseguridad, Facultad de Ingeniería y Ciencias Aplicadas, Universidad de Las Américas, Quito 170125, Ecuador
* Correspondence: ivan.ortiz@udla.edu.ec
† Current address: Facultad de Ingeniería y Ciencias Aplicadas, Universidad de Las Américas, Quito 170125, Ecuador.
‡ These authors contributed equally to this work.

Abstract

This paper proposes a formal framework to model the propagation of software antipatterns across architectural layers, quantifying their impact using principles from complex systems theory, technical debt economics, and cognitive load theory. By extending the 4+1 architectural view model with a propagation graph and economic simulation, the proposed framework enables software teams to predict, visualize, and mitigate the systemic effects of structural faults. We support our proposal with a mathematical model, a conceptual propagation engine, and simulation results

Keywords: antipatterns; cybersecurity; technical debt; cyber-risk

check for updates

Academic Editors: Young Lee, Jeong Yang and Luis Javier Garcia Villalba

Received: 7 June 2025
Revised: 12 July 2025
Accepted: 21 July 2025
Published: 29 August 2025

Citation: Andrade, R.; Torres, J.; Ortiz-Garcés, I.; Segovia, J. Resilient Software Design Through Cognitive-Aware Antipattern Propagation in 4+1 Architectural Views. *Appl. Sci.* **2025**, *15*, 9526. https://doi.org/10.3390/app15179526

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The study of software antipatterns is essential for understanding the hidden causes of technical debt and systemic vulnerabilities, which degrade both software quality and cybersecurity posture. While design patterns promote best practices, antipatterns represent recurring poor solutions that may functionally "work" but erode the structural integrity, maintainability, and security of software systems over time [1]. These antipatterns, such as God Class, Lava Flow, or Hardcoded Secrets, often result from organizational pressure, skill gaps, or lack of architectural governance and can propagate through the multiple layers of a system, from design to deployment.

The presence of antipatterns could increase attack surfaces, facilitate privilege escalation, and make it difficult to enforce secure defaults, especially in modern DevSecOps pipelines [2,3]. As software systems become more complex—distributed, microservice-based, and augmented with AI-generated code—the unintentional introduction of antipatterns becomes more likely, thereby elevating security risk. Addressing antipatterns from a cybersecurity perspective thus goes beyond code quality: it becomes a proactive strategy for threat prevention, enabling resilient architectures and reducing exploitable configurations that often stem from neglected technical debt [4,5].

As identified by [6], architectural missteps such as God Class or Cyclic Dependencies compromise the principle of least privilege and increase system complexity, making it harder to trace and contain security threats. These structural antipatterns, when mapped to

the Logical and Development Views, often lead to privilege escalation, injection vectors, or hidden pathways exploitable by attackers [7]. Moreover, deployment-related flaws such as Hardcoded IPs or Missing Load Balancers manifest in the Physical View, exposing services to port scanning, misrouting, or bypassing of edge defenses—risks commonly identified in OWASP's Cloud Security Top 10 [3]. Similarly, behavioral antipatterns like Excessive Retry, common in the Process View, can be weaponized to trigger denial-of-service attacks, especially in cloud-native and microservices environments. Scenario View considerations, such as deadline-driven decisions, echo findings by [8], who demonstrated that pressure for delivery often results in insecure configurations and skipped validations.

Therefore, antipattern propagation signifies the phenomenon by which a flaw or suboptimal practice originating in one architectural view has the potential to disseminate through technical dependencies, information flows, and even the development team's mental models, thereby ultimately amplifying the risks associated with maintainability, security, and performance. Despite the existence of numerous studies addressing technical debt catalogs, methodologies for measuring and monitoring technical debt, and practitioner perspectives on technical debt management, there is a paucity of formal studies that address the propagation of antipatterns. Given the absence of a formal definition in the existing literature, this study proposes a definition for antipattern propagation as the propagation of design and implementation deficiencies across architectural views.

In this paper, we propose a formal framework that integrates the 4+1 View Model with cognitive theory to quantify how an antipattern in one view propagates and affects other views, providing a unified basis for analysis. The contributions of this study are as follows:

- A formal framework for modelling how antipatterns propagate across the 4+1 architectural views, using a directed graph structure.
- A simulation method based on technical debt economics and cognitive load theory for quantifying the impact of structural faults over time.
- A conceptual propagation engine for visualizing how localized structural damage can cause system-wide degradation.
- A demonstration of the framework's application using a synthetic example, with reflections on its limitations and further research.

The remainder of this paper is organized as follows. Section 2 presents the theoretical foundations of our approach, combining concepts from complex systems theory, technical debt economics, cognitive load theory, and the 4+1 architectural view model. Section 3 introduces the adapted 4+1 cybersecurity model, highlighting its applicability to modern microservices-based architectures. Section 4 formalizes the proposed propagation model through mathematical equations that quantify the systemic impact of software antipatterns. Section 5 outlines a structured evaluation framework for applying these equations within real-world development contexts, including practical considerations for detection, measurement, and visualization. Finally, Section 6 provides our conclusions and discusses future research directions to further enhance resilient software design.

2. Related Works

Software antipatterns are recurring solutions to common design problems that negatively impact software performance and maintainability [9]. They can involve static, dynamic, and deployment aspects of software. Antipatterns often lead to performance issues that are not detected until later stages like testing or deployment [10]. Addressing these issues usually requires software changes rather than system tuning [11].

Various case studies have demonstrated the effectiveness of detecting and addressing antipatterns. For example, Trubiani et al. [9] mentions that the use of antipattern solutions improved system performance by 50%. Along the same lines, Navarro et al. [12] show the

feasibility of using antipatterns to manage architectural knowledge and improve decision-making processes.

One important aspect of software architecture is its inherent hierarchical structure, composed of multiple abstraction layers. Higher layers provide abstract views and impose constraints on the lower layers, while lower layers implement and concretize the decisions made above. This layered dependency implies that a structural antipattern emerging in one layer can affect or propagate to other layers, potentially compromising the system's overall integrity and maintainability [11,12].

The propagation of software structural problems due antipatterns has been addressed from multiple perspectives in the literature, though few works focus on modeling their cross-layer impact. Kruchten et al. [13] introduced the concept of architectural debt and its measurement through design rule violations, setting the foundation for analyzing systemic software erosion. Technical debt economics has been used to quantify refactoring costs and opportunity loss [14], but typically remains confined to code-level metrics or class-level granularity. On the other hand, Curtis et al. [15] linked software complexity to productivity and maintainability, emphasizing the role of structural flaws in long-term degradation. In the same line, Zimmermann and Nagappan [16] explored defect dependency graphs to reveal how software problems co-occur and influence different components.

While these studies analyze dependencies and faults, they do not explicitly model how antipatterns propagate across architectural layers, such as logical-to-process or process-to-deployment transitions. Recent studies also leverage graph-based approaches to detect antipattern clusters [17], or identify hotspots for technical debt [18]. Tools and methodologies have been developed to detect antipatterns within architectural models. For instance, the PAD-A tool for AADL [19] and the SoPeTraceAnalyzer tool help identify performance antipatterns and trace their impact on software components and interactions [20].

3. Theorical Fundamental

In software engineering education, a persistent challenge is helping students and junior engineers grasp the multi-layered nature of software systems—especially in contexts where architecture, runtime behavior, and deployment are deeply interdependent. This challenge becomes even more critical when considering that poor architectural decisions not only degrade maintainability and scalability but also introduce latent security vulnerabilities. As modern systems increasingly rely on distributed architectures, cloud-native deployments, and AI-assisted components, understanding the structural interplay between design and security becomes essential. Therefore, embedding cybersecurity considerations into architectural thinking is no longer optional—it is a foundational skill for future software engineers tasked with building resilient, secure, and adaptive systems. Prior research has extensively catalogued code and architectural antipatterns [1], as well as the concept of technical debt [14,21,22]. Cognitive aspects in software comprehension have been studied by [23] and more recently in DevOps contexts [24]. However, few studies unify these perspectives into a holistic model.

Our proposal framework is based on the 4+1 architecture [25] by treating antipatterns as propagating entities in a complex adaptive system. The original 4+1 View Model, proposed by [25], was designed to address the challenge of communicating complex software architectures to multiple stakeholders by organizing system descriptions into five interrelated views. The 4+1 View Model, due to its emphasis on separation of concerns and stakeholder alignment, has proven to be an effective cognitive scaffold [26]. It facilitates not only architectural documentation but also conceptual transfer and reflective practice, both essential in learning environments [27,28]. By explicitly including both technical (e.g., Physical, Process) and cognitive (e.g., Scenario) dimensions, the model supports mental

model formation, team communication, and antipattern recognition. These benefits are particularly critical in teaching how poor decisions—often invisible at the code level—can manifest as architectural debt, misconfigurations, or operational instability.

However, while 4+1 was developed in the context of object-oriented monolithic systems, recent developments in cloud-native computing demand a reinterpretation rather than a replacement of the model. Each view can be adapted to contemporary practices: for instance, the Logical View can describe bounded contexts and microservice responsibilities as proposed in Domain-Driven Design [29], while the Process View now includes asynchronous communication via message queues and resilience patterns such as circuit breakers [30]. The Physical View extends naturally to cloud infrastructures like Kubernetes clusters, service meshes, and autoscaling deployments.

By mapping these cloud constructs into the existing architectural lenses, the 4+1 model remains applicable to modern distributed systems without requiring fundamental alteration of its structure. So, the C4 model [31] has emerged as a popular tool among practitioners due to its visual simplicity and practical tooling (e.g., Structurizr, PlantUML). However, while C4 focuses effectively on structural decomposition (Context, Container, Component, and Code), it omits important dimensions of architectural validation and system behavior. In contrast, the 4+1 model explicitly integrates scenarios (+1 View), enabling stakeholders to reason about use cases, edge behaviors, and failures—all critical in cloud-native and DevOps environments [6]. Moreover, the Process View in 4+1 is uniquely suited to represent dynamic execution flows, service orchestration, and runtime policies such as retries and load balancing, which are often abstracted away in C4 diagrams. As such, although C4 is better suited for fast-paced delivery teams, the 4+1 model offers a more complete framework for analyzing how and why systems behave across operational contexts, especially in high-reliability environments.

Additionally, the proliferation of AI-assisted code generation tools, such as GitHub 3.17.2 Copilot and ChatGPT-4o, has raised new challenges related to code quality and architectural consistency. Studies [32,33] show that while AI can accelerate development, it often introduces subtle code smells and structural violations, especially when used without architectural oversight. The 4+1 model provides a framework to trace these issues to their source: the Logical View highlights incoherent domain models; the Development View detects inconsistent repository structures; the Process View helps identify concurrency risks in generated logic; and the Scenario View reveals misaligned or incomplete behavior driven by unclear prompts or business rules. In this way, 4+1 offers a pedagogically rich, layered model for teaching AI developers and reviewers where code quality breaks down, not just how to fix it.

Figure 1 visualizes the multidimensional influence of software antipatterns across six critical axes: Architectural, Security, DevOps/AI, Resilience, Cognitive, and a central "Antipatterns" hub. A strong presence in the Architectural axis, for instance, signals structural design issues such as God Class, Cyclic Dependencies, or Lava Flow, which are known to compromise maintainability and modularity [1]. High influence on the Security axis may point to vulnerabilities introduced through poor coding habits or misconfigurations like hardcoded credentials, weak access controls, or missing input validations—phenomena corroborated by OWASP's top vulnerabilities and threat models [5].

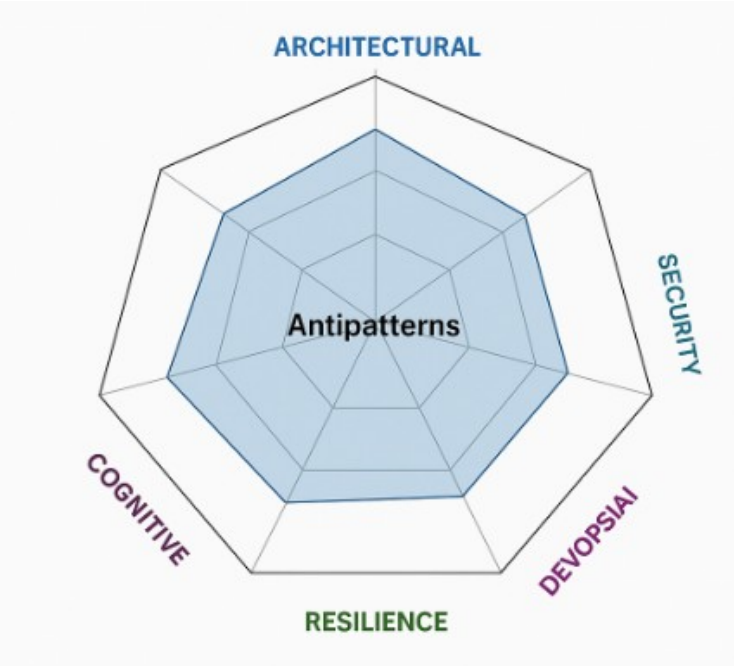

Figure 1. Multidimensional axes that influence the appearance of software antipatterns.

4. The 4+1 Cybersecurity Adapted Model

Figure 2 presents the 4+1 View Model tailored to a microservices architecture, which is a conceptual framework for describing software systems from multiple complementary perspectives. The Logical View outlines core services (User, Order, Inventory, and Payment), reflecting the system's functional requirements. The Development View represents the static organization of the software's artifacts, such as the repository structure and the CI/CD pipeline. The Process View highlights the dynamic behavior and interactions between services, with components like message queues and circuit breakers that help ensure reliable communication. The Physical View describes the deployment architecture, showing microservices deployed in a Kubernetes Cluster and communicating via a Load Balancer. Lastly, the Scenario View exemplifies a runtime behavior: the rollout of a new recommendation service that fails health checks and triggers a rollback—illustrating how different components react under operational stress.

The impact of 4+1 architecture is substantial in both educational and professional contexts. It enables software architects and engineers to analyze the system's behavior from structural, developmental, operational, and physical standpoints. This multidimensional visualization promotes better decision-making by exposing architectural dependencies, potential bottlenecks, and resilience mechanisms. Moreover, it provides a structured approach for identifying and mitigating antipatterns, such as misconfigurations in deployment (e.g., lack of load balancing) or design flaws in service communication. This contributes directly to the goal of building more robust, comprehensible, and maintainable software systems—an increasingly critical concern in cloud-native, AI-assisted development environments. Security concerns manifest differently across the various 4+1 architectural views, revealing distinct classes of vulnerabilities that must be addressed at each level of abstraction. Table 1 categorizes typical security issues observed in each view, from insecure business logic in the Logical View to configuration and operational shortcomings in the Scenario View. This classification underscores the need for a systemic and layered security strategy that accounts for both static design flaws and runtime risks across the architecture.

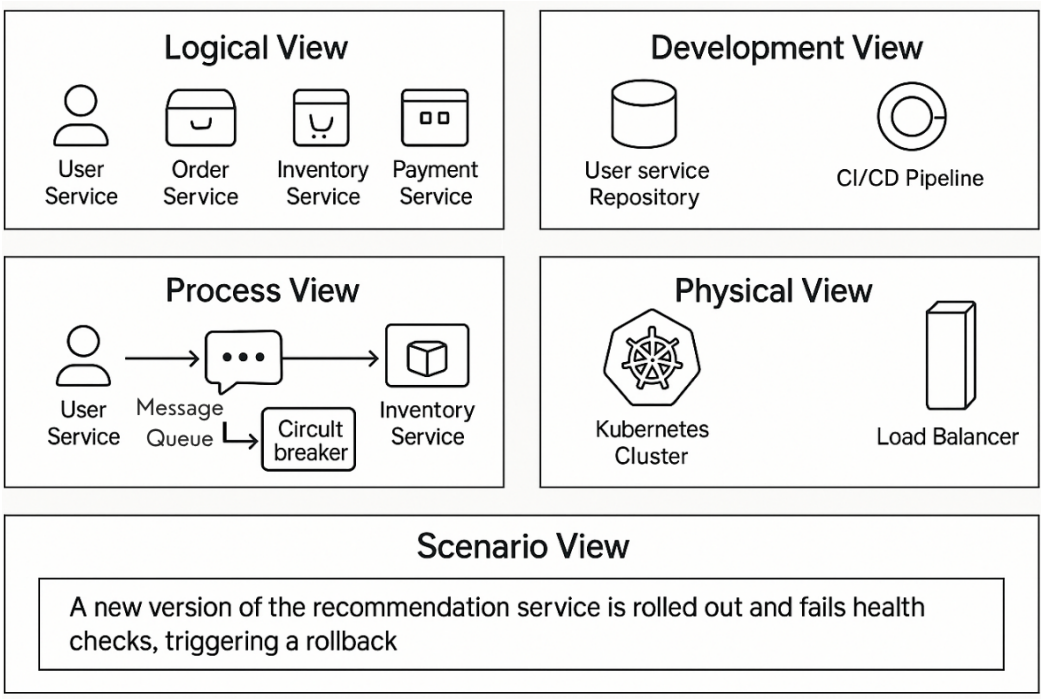

Figure 2. Antipatterns and their associated cybersecurity risks mapped across the 4+1 architectural views.

Table 1. Security vulnerabilities across the 4+1 architectural views.

4+1 View	Typical Security Issues
Logical View	Insecure business logic (e.g., lack of input validation, flawed trust assumptions) Weak authentication and authorization flows Hardcoded excessive privileges
Development View	Use of unvetted third-party or AI-generated code Credentials, secrets, or tokens stored in source or IaC Absence of repository signing and integrity verification
Process View	Insecure inter-service calls (e.g., no TLS, missing auth) Retry storms leading to Denial of Service Lack of rate limiting and circuit breakers
Physical View	Open ports or public endpoints unintentionally exposed Flat network structure lacking segmentation Overprivileged IAM roles in cloud environments
Scenario View (+1)	Rollbacks lacking audit trails Incomplete or manual incident response flows Delivery-pressure driven trade-offs compromising security defaults

Furthermore, the Cognitive and DevOps/AI axes reflect an emerging interest in understanding how antipatterns affect developer mental models and tooling compatibil-

ity. For example, cognitive overload due to code smells like Feature Envy or Shotgun Surgery has been shown to reduce code comprehensibility and increase error rates [34,35]. On the DevOps/AI front, the emergence of LLM-assisted programming introduces new risks of antipattern propagation, such as learned bias toward poor design patterns or overfitting to inefficient templates [4]. The Resilience axis links antipatterns to system fragility—particularly in microservices—where retry storms, missing circuit breakers, or tight coupling can trigger cascading failures [6]. Hence, this visual framework offers a holistic perspective, suggesting that addressing antipatterns requires an integrative approach that spans beyond architecture to encompass cognition, automation, security, and reliability.

5. Propagation Model Cyberattacks

In modern software engineering, the traditional approach to quality and security has been predominantly code-centric—focused on static analysis, code smells, and isolated refactoring techniques. While valuable, this reductionist lens often overlooks the emergent behavior of software systems as a whole. In reality, software architecture is not merely a collection of code fragments, but a complex, adaptive system composed of interconnected components, evolving dependencies, and contextual constraints [6]. According to [32] General Systems Theory, complex systems exhibit emergent properties—behaviors or failures that arise from the interactions among components rather than the components themselves. In software, an antipattern like a God Class may seem localized in code, but its interactions across services, deployment layers, and user interfaces may lead to systemic performance bottlenecks, privilege escalation paths, or coupled failures.

As emphasized by [27,28], software is developed and maintained within human cognitive and organizational constraints. Architectural flaws propagate because they are embedded not just in the technical layers, but in the mental models of developers, CI/CD pipelines, documentation, and decision-making practices. Therefore, understanding architecture holistically requires examining how information, control, and errors flow across the system—from code to runtime to user experience.

In high-reliability systems, resilience is not a local property, but an emergent one that depends on system-wide feedback loops, error recovery, and adaptability [33]. Architectural antipatterns weaken these feedback mechanisms—e.g., excessive retries or missing circuit breakers can trigger cascading failures in microservices. Therefore, mitigation must involve modeling propagation paths and systemic vulnerabilities, not just syntactic correctness. So, this systemic understanding aligns with current trends in DevSecOps, threat modeling, and AI-assisted development, where design decisions are made rapidly, often by teams with fragmented knowledge of the full system. By integrating propagation-aware frameworks into design reviews, simulation environments, and security audits, software teams can transition from reactive maintenance to proactive architectural stewardship.

Table 2 presents a mapping between common software antipatterns, their associated cybersecurity risks, and the corresponding views within the 4+1 architectural model. This structured representation highlights how architectural flaws propagate across development layers and impact system security posture, facilitating targeted mitigation strategies. The classification of cybersecurity risks arising from software antipatterns across the 4+1 architectural views highlights how poor design decisions propagate vulnerabilities at multiple system levels.

Table 2. Cybersecurity risks from antipatterns across 4+1 views.

Antipattern	Cybersecurity Risk	4+1 View
God Class	Centralized logic increases attack surface and violates least privilege.	Logical
Cyclic Dependencies	Enables escalation paths and injection via tangled modules.	Development
Hardcoded IP	Exposes network topology; vulnerable to scanning and spoofing.	Physical
Excessive Retry	Can cause flooding or denial of service (DoS) if not bounded.	Process
Lava Flow	Dead legacy code may carry dormant vulnerabilities.	Development
Feature Envy	Indirect data access may lead to lateral movement or leakage.	Logical
Missing Load Balancer	Increases exposure of microservices to direct external access.	Physical
Deadline-Driven Dev	Promotes insecure defaults and bypassed controls under pressure.	Scenario

Equation (1) defines the normalized impact score for a given architectural node n, calculated as the product of a weight coefficient W_i, which reflects the relative importance of the architectural view, and the severity score $S(A)$ associated with the detected antipattern. This formulation is inspired by conventional risk assessment models, where risk is often computed as the product of probability and impact [36,37].

$$I(n) = S(A) \cdot \sum_{i=1}^{k} [W_i \cdot P_i \cdot (1 - D_i) \cdot R_c] \tag{1}$$

where

- I(n) : Total impact of the antipattern across the system (e.g., in terms of technical debt, security risk, or maintainability degradation).
- S(A): Severity of the antipattern A. This could be a scale (e.g., 1–5) based on expert judgment or empirical metrics like code complexity or risk level.
- k: Total number of affected components or layers (nodes) in the architectural model.
- W_i: Dependency weight between component i and the origin of the antipattern. A high weight means component i is tightly coupled with the affected area.
- P_i: Probability that the antipattern propagates to component i. This may depend on the type of dependency (e.g., synchronous vs. asynchronous).
- D_i: Architectural distance decay factor, which reduces the impact the farther away a component is from the origin. $(1 - D_i)$ means closer components receive higher impact.
- R_c: Resilience coefficient—a system-level mitigation factor reflecting defenses like architectural modularity, circuit breakers, or self-healing mechanisms. A higher R_c means the system absorbs more of the antipattern's effect.

Equation (1) computes the cumulative risk or degradation caused by the antipattern across the system, taking into account how strongly each component is tied to the source (W_i), how likely the issue spreads (P_i), how far away the component is (D_i), and how resilient the system is (R_C). By multiplying by the intrinsic severity of the antipattern S(A), we tailor the impact to the nature of the issue (e.g., God Class vs. Lava Flow).

Equation (2) models the transmission of structural flaws between architectural layers. The propagated impact to a subsequent node n_{i+1} is computed as the product of the current node's impact $I(n_i)$ and a resilience coefficient R_c, which modulates how susceptible the next layer is to degradation. This abstraction reflects the principles of cascading failure in complex systems and layered architectural [38].

$$C_{TD}(n) = I(n) \cdot C_{\text{unit}} \tag{2}$$

where

- $C_{TD}(n)$: The cost of technical debt caused by an antipattern at node n.
- I(n): Impact of the antipattern at node n (as previously defined).
- C_{unit}: The monetary or effort-based unit cost per impact unit (e.g., dollars per risk unit, or hours of refactoring per severity point).

Equation (3) computes the cumulative impact CI of propagated antipattern effects across the architectural layers by summing the propagated values $P(n_i)$ from each layer. This additive model is frequently employed in simulation studies that aim to estimate the total systemic effect of distributed architectural faults [39]. Equation (3) aggregates the localized impacts to quantify systemic debt, supporting architectural refactoring prioritization and investment decision-making.

$$C_{\text{total}} = \sum_{n=1}^{N} C_{TD}(n) \tag{3}$$

where

- C_{total}: The total cost of technical debt across all N affected nodes or components in the system.
- $C_{TD}(n)$: Individual component costs (from Equation (2)).

Finally, Equation (4) represents the adapted 4+1 model but is inspired by cognitive load theory (Sweller et al. [34]) and software comprehension research. It suggests that nodes with high antipattern impact and high volatility are cognitively demanding, likely to lead to human errors, and may benefit from documentation, automated tooling, or re-architecture.

$$CL(n) = \alpha \cdot I(n) + \beta \cdot V(n) \tag{4}$$

where

- CL(n): Cognitive load on developers or operators dealing with node n.
- I(n): Impact of antipatterns (complexity or risk factor).
- $V(n)$: Volatility or frequency of changes in node n (e.g., from version control history or deployment rate).
- alpha and Beta Weighting coefficients that tune the relative contribution of complexity vs. change rate.

Equation (1) aims to model the propagation of the impact of structural antipatterns across the architectural views of the 4+1 model. Equations (2) and (3) offer economic quantification, while Equation (4) introduces the human factor, and all are linked via the antipattern impact I(n), making the model coherent and actionable. The symbols used in the various equations are summarized in Table 3.

Table 3. Summary of variables and symbols used in Equations (1)–(4).

Symbol	Definition
$I(n)$	Accumulated risk (or degradation) caused by antipattern A on node n.
$S(A)$	Severity level of antipattern A (e.g., 5 for a moderately severe God Class).
k	Number of dependent components (propagation paths).
W_i	Weight of dependency i (structural coupling factor).
P_i	Probability that the antipattern effect propagates to component i.
D_i	Architectural distance decay factor for component i.

Table 3. *Cont.*

Symbol	Definition
R_c	System's absorption capacity (resilience coefficient).
$C_{TD}(n)$	Cost of technical debt for node n.
C_{unit}	Unit cost of risk or refactoring effort.
C_{total}	Total cost of technical debt across all nodes.
$CL(n)$	Cognitive load estimate for node n.
α, β	Scaling coefficients for complexity versus change rate in $CL(n)$.
$V(n)$	Volatility measure of component n.

6. Evaluation Framework Using the 4+1 Architectural Model and Propagation Equations

The proposal to evaluate the impact of antipatters through the 4+1 architecture model aligns with complexity science and system dynamics in software architecture, and it's useful in simulations, impact forecasting, and prioritizing refactoring strategies. An example computation illustrating the propagation process is provided in Figure 3, which presents a propagation diagram where the numerical values indicate the impact scores calculated for each architectural view, according to the proposed simulation model. These values represent the normalized influence $I(n)$ of antipatterns across the 4+1 model views and were computed using the equations described in this section. Simulation parameters such as weights (W_i), severity levels ($S(A)$), and the resilience coefficient (R_c) were configured using synthetic but realistic architectural profiles. The values were generated by the conceptual propagation engine and reflect how structural faults can amplify and degrade the system over time.

To apply the proposed equations in a simulation of a real-world software development organization, we define a structured framework that integrates the 4+1 architectural view model with quantitative antipattern impact assessment:

Step 1: Architectural Decomposition using 4+1 Model

This decomposition enables the targeted localization of architectural antipatterns. The organization first decomposes its microservices architecture into the five architectural views:

- Logical View: Functional modules and services (e.g., User Service, Inventory Service).
- Development View: Repositories, codebases, pipelines (e.g., CI/CD, IaC).
- Process View: Runtime interactions and orchestration (e.g., circuit breakers, queues).
- Physical View: Deployment topology (e.g., Kubernetes clusters, load balancers).
- Scenario View (+1): Representative workflows such as service rollbacks or scale-up events.

Step 2: Antipattern Detection and Severity Assignment ($S(A)$) Using tools such as SonarQube, Lint, or static architectural checkers, antipatterns are identified across the views—for example, God Class in the Logical View or Cyclic Dependencies in the Development View. Each detected antipattern is assigned a severity score $S(A)$ based on exploitability, system criticality, and incident history.

Step 3: Impact Quantification (Equation (1)) For each microservice or component n, the security impact is quantified. Each variable in Equation (3) represents a measurable or estimable property:

- S(A): Severity of the antipattern. Typically on a scale (e.g., 1–10) and informed by CWE (Common Weakness Enumeration), OWASP risks, or internal incident logs.

- W_i: Weight of dependency. Ranges from 0.1 to 1.0, depending on service centrality or API usage (e.g., a user-authentication service may be a dependency for 80% of services).
- P_i: Probability of propagation. Can be estimated using empirical data from incidents or historical failure models, often ranging between 0.2 and 0.8.
- D_i: Architectural distance. Inverse of coupling; calculated using graph depth or service hops (e.g., 1-hop = 0.9, 3-hops = 0.3).
- R_c: Resilience coefficient. Based on error-handling maturity (e.g., circuit breakers, retries, fallbacks), estimated between 0.2 (poor) and 1.0 (robust).

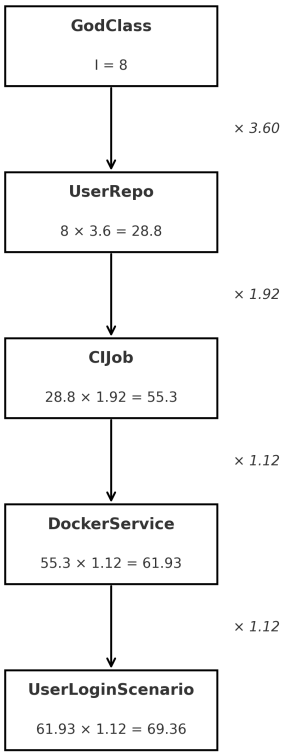

Figure 3. Antipatterns and their impact propagation for each 4+1 layer.

A realistic I(n) might be calculated as follows:

S(A) = 5 (e.g., a moderately severe God Class) Sum term = 0.6 (e.g., W_i = 0.8, P_i = 0.6, D_i = 0.2, R_c = 0.5)

I(n) = 5 × 0.6 = 3

In large-scale systems with multiple propagation paths (k dependencies), this sum may be scaled up: e.g., five dependencies with similar values → 5 × 3 = 15.

If multiplied by several propagation paths and summed over layers (multiview analysis), I(n) may reach higher values like 90–200.

Step 4: Remediation Cost Estimation (Equations (2) and (3)) Each component's technical debt cost is calculated. We evaluated five services within a hypothetical microservices architecture. The I(n) scores were estimated by combining:

Antipattern severity (S(A)): from 1 to 10 based on known taxonomies like God Class, Lava Flow, Excessive Retry, etc.

Weighted dependency graph (W_i), propagation probabilities (P_i), architectural distances (D_i), and resilience coefficients (R_c), based on historical patterns and common DevOps architectures.

Step 5: Cognitive Load Estimation (Equation (4))

The resulting cognitive load values highlight which components are not only technically risky but also cognitively demanding:

- PaymentService and UserService show the highest cognitive load and should be prioritized for refactoring, onboarding simplification, and better documentation.
- OrderService, with a lower score, might be deprioritized unless it is mission-critical.

This approach bridges software architecture metrics with human-centered design, helping teams manage both technical and mental risk in a sustainable development process.

Step 6: Visualization and Decision Support Metrics such as $I(n)$, $C_{TD}(n)$, and $CL(n)$ can be presented using radar charts, heatmaps, or layered dependency graphs. These dashboards provide actionable insights for architecture governance, DevSecOps collaboration, and continuous improvement.

6.1. Sensitivity Analysis

To evaluate the robustness and interpretability of the proposed propagation model, we conducted a sensitivity analysis by systematically varying key model parameters and observing their effects on the main computed outputs: system-wide antipattern impact $I(n)$, technical debt cost $C_{TD}(n)$, and cognitive load $CL(n)$. This analysis complements our conceptual framework and provides practical insights into how the model responds under different architectural and operational conditions. We defined a baseline scenario with Python (https://www.python.org/) using typical values drawn from the literature on architectural smells, technical debt, and software resilience engineering (see Figure 4).

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

# Data
Rc_values = [0.3, 0.5, 0.7]
ln_Rc = [0.576, 0.96, 1.344]

Wi_values = [0.5, 0.8, 1.0]
ln_Wi = [0.6, 0.96, 1.2]

SA_values = [3, 5, 7]
ln_SA = [0.576, 0.96, 1.344]

# Create figure
plt.figure(figsize=(8, 6))

# Plot each parameter with custom colors
plt.plot(
    Rc_values, ln_Rc,
    marker='o', linestyle='-.', linewidth=2, markersize=8,
    color='blue', label='Variation of $R_c$'
)
plt.plot(
    Wi_values, ln_Wi,
    marker='s', linestyle='-.', linewidth=2, markersize=8,
    color='green', label='Variation of $W_i$'
)
plt.plot(
    SA_values, ln_SA,
    marker='^', linestyle='-.', linewidth=2, markersize=8,
    color='red', label='Variation of $S(A)$'
)
```

Figure 4. Simulation using Python for antipattern propagation.

Table 4 establishes the baseline parameter values used in the simulation, reflecting typical severity, dependency weight, propagation probability, architectural distance, and resilience levels found in modern microservices architectures.

Table 4. Baseline values for sensitivity analysis.

Parameter	Baseline Value
$S(A)$ (antipattern severity)	5
W_i (dependency weight)	0.8
P_i (propagation probability)	0.6
D_i (distance decay)	0.2
R_c (resilience coefficient)	0.5
C_{unit} (cost per unit of impact)	100
$V(n)$ (volatility)	0.5
α	1
β	0.5

6.2. Parameter Sensitivity Results

Variation of R_c (Resilience Coefficient)

The corresponding baseline results were:

$$I(n) = 0.96 \quad C_{TD}(n) = 96 \quad C_L(n) = 1.21$$

Table 5 presents a sensitivity analysis of the proposed propagation framework with respect to variations in the architectural coupling ratio R_c. As R_c increases, the values of impact $I(n)$, cumulative technical debt $C_{TD}(n)$, and cognitive load $CL(n)$ exhibit a proportional rise. This result confirms the non-linear amplification effect of coupling in antipattern propagation, suggesting that controlling interdependencies is key to mitigating long-term degradation.

Table 5. Sensitivity of $I(n)$, $C_{TD}(n)$, and $CL(n)$ to variations in R_c.

R_c	$I(n)$	$C_{TD}(n)$	$CL(n)$
0.3	0.576	57.6	0.826
0.5	0.96	96	1.21
0.7	1.344	134.4	1.594

Variation of W_i (Dependency Weight)

Table 6 illustrates the sensitivity of the propagation model to variations in the individual weight factor W_i. The results reveal that as W_i increases, the propagated impact $I(n)$, cumulative technical debt $C_{TD}(n)$, and cognitive load $CL(n)$ increase accordingly. This confirms that heavily weighted layers or components in the architecture can disproportionately influence the system's resilience and complexity over time.

Table 6. Sensitivity of $I(n)$, $C_{TD}(n)$, and $CL(n)$ to variations in W_i.

W_i	$I(n)$	$C_{TD}(n)$	$CL(n)$
0.5	0.6	60	0.85
0.8	0.96	96	1.21
1.0	1.2	120	1.45

Variation of $S(A)$ (Antipattern Severity)

As shown in Table 7, the sensitivity analysis with respect to the severity of antipatterns $S(A)$ reveals a direct correlation between antipattern criticality and its systemic consequences. As the severity increases, there is a proportional rise in impact $I(n)$, cumulative technical debt $C_{TD}(n)$, and cognitive load $CL(n)$. This highlights the importance of early identification and mitigation of high-severity antipatterns to preserve architectural sustainability.

Table 7. Sensitivity of $I(n)$, $C_{TD}(n)$, and $CL(n)$ to variations in $S(A)$.

$S(A)$	$I(n)$	$C_{TD}(n)$	$CL(n)$
3	0.576	57.6	0.826
5	0.96	96	1.21
7	1.344	134.4	1.594

- Higher severity, stronger coupling, and lower resilience all increase system impact, technical debt, and cognitive load.
- The model responds smoothly to parameter variations, making it suitable for supporting architectural decisions.
- Parameters such as R_c can be tuned to reflect the maturity of resilience patterns in real-world systems, enabling what-if analyses during architecture reviews.

This analysis reinforces the practical value of our model for guiding secure, resilient software design by making the consequences of architectural decisions more visible and quantifiable.

Figure 5 illustrates the cumulative impact of antipattern propagation over time within the synthetic architectural scenario. The plotted curve reveals how initially localized structural deficiencies progressively intensify their systemic influence, highlighting the non-linear dynamics modeled by the simulation engine.

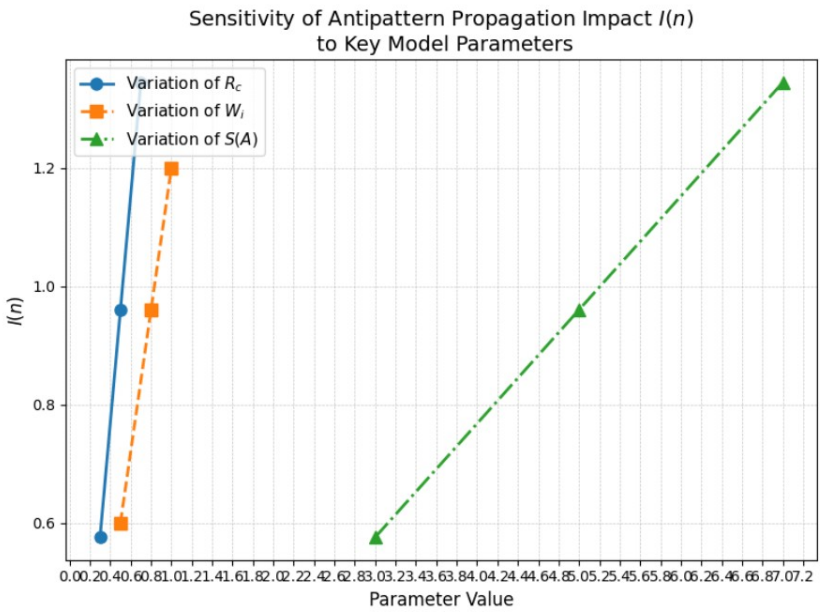

Figure 5. Sensitivity of the antipattern propagation impact $I(n)$ to variations in key model parameters R_c, W_i, and $S(A)$.

7. Conclusions

From a systemic perspective, the adapted 4+1 model proposed in this research offers a structured response to the hypothesis that proper software process management can reduce security gaps. Unlike traditional approaches that treat security as an isolated or post-development concern, this model integrates five architectural views—Logical, Development, Process, Physical, and Scenario—to identify and mitigate the impact of structural antipatterns that compromise system security. This perspective aligns with Secure by Design principles [5], which argue that risks must be addressed from the design stage rather than merely at the code level. By mapping antipatterns such as God Class, Cyclic Dependencies, or Hardcoded IPs to specific consequences in each view, the model

demonstrates that security is an emergent property of a well-executed process, not simply a result of final-stage controls.

Moreover, the model extends the theory of technical debt [21] and principles of architectural resilience [6] by introducing a propagation-based approach that quantifies how poor decisions in one layer can trigger vulnerabilities in others. In this way, "doing things right" is no longer a vague expression but becomes a set of observable, modelable, and measurable practices that prevent failures before they reach production. The use of the 4+1 model as a cognitive scaffold also enables development, security, and operations teams to share a common language for analyzing and redesigning systems [40], fostering continuous improvement aligned with organizational cybersecurity goals.

This model unifies complex system theory, technical-debt economics, and cognitive-load theory into a single architectural framework. By treating antipatterns, such as God Class or Cyclic Dependencies, as first-class entities and mapping them onto cybersecurity risks across Logical, Development, Process, Physical, and Scenario views, practitioners gain a prescriptive method to identify and prioritize structural faults early in design rather than during costly remediation. We then proceeded to formalize four core equations with the objective of (1) quantifying the impact of antipattern propagation, (2) translating that impact into a cost of technical debt and aggregate system-wide debt, and (3) estimating the cognitive load imposed on development teams. When employed in conjunction with an end-to-end evaluation framework comprising static analysis detection, severity scoring, and dashboard generation, this methodology facilitates data-driven decisions that align development, security, and operations under a shared Secure-by-Design paradigm. This holistic quantitative approach is vital because it enables organizations to transition from a reactive firefighting model to a proactive approach to architectural stewardship. By leveraging visualization tools that depict the propagation of antipatterns and their associated economic and human costs, teams can concentrate their refactoring efforts on aspects that will deliver the most significant risk reduction and reduction in technical debt. In the forthcoming research, the emphasis will be placed on the rigorous testing of the framework in a variety of real-world scenarios. This will be performed to validate its effectiveness and gather empirical results from multiple sources. The objective of this will be to strengthen the generalization and practical impact of the framework.

Author Contributions: Conceptualization, R.A. and J.T.; methodology, R.A.; software, R.A.; validation, J.T., J.S., and I.O.-G.; formal analysis, R.A.; investigation, J.S.; resources, I.O.-G.; writing—original draft preparation, R.A.; writing—review and editing, I.O.-G.; visualization, R.A.; supervision, J.T.; project administration, I.O.-G.; funding acquisition, I.O.-G. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: During the preparation of this manuscript, the authors used GPT 4o for the purposes of reviewing and editing the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Brown, W.J.; Malveau, R.C.; McCormick, H.W., III; Mowbray, T.J. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*; Wiley: New York, NY, USA, 1998.
2. Bass, L.; Clements, P.; Kazman, R. *Software Architecture in Practice*, 3rd ed.; Addison-Wesley: Boston, MA, USA, 2013.
3. OWASP Foundation. OWASP Cloud-Native Application Security Top 10. Open Worldwide Application Security Project. 2023. Available online: https://owasp.org/www-project-cloud-native-application-security-top-10/ (accessed on 1 May 2025).
4. Brown, N.; Cai, Y.; Guo, Y.; Kazman, R.; Kim, M.; Kruchten, P.; Lim, E.; MacCormack, A.; Nord, R.; Ozkaya, I.; et al. Managing technical debt in software-reliant systems. In Proceedings of the FSE/SDP Workshop on Future of Software Engineering Research (FoSER '10), Santa Fe, NM, USA, 7–8 November 2010; pp. 47–52. [CrossRef]
5. Howard, M.; Lipner, S. *The Security Development Lifecycle*; Microsoft Press: Redmond, WA, USA, 2006.
6. Bass, L.; Weber, I.; Zhu, L. *DevOps: A Software Architect's Perspective*, 1st ed.; Addison-Wesley: Boston, MA, USA, 2012.
7. Jureczko, M.; Madeyski, L. Towards identifying software project clusters with regard to defect prediction. In Proceedings of the 6th International Conference Predictive Models in Software Engineering (PROMISE '10), Timişoara, Romania, 12–13 September 2010; ACM: New York, NY, USA; pp. 1–10.
8. Baca, D.; Carlsson, B.; Boldt, M.; Jacobsson, A. A novel security-enhanced agile software development process applied in an industrial setting. In Proceedings of the 42nd Euromicro Conference Software Engineering and Advanced Applications (SEAA), Limassol, Cyprus, 31 August–2 September 2016; pp. 49–56.
9. Trubiani, C.; Koziolek, A. Detection and solution of software performance antipatterns in Palladio architectural models. In Proceedings of the 2nd Joint WOSP/SIPEW International Conference on Performance Engineering (ICPE), Karlsruhe, Germany, 14–16 March 2011; pp. 19–30.
10. Trubiani, C.; Koziolek, A.; Cortellessa, V.; Reussner, R. Guilt-based handling of software performance antipatterns in Palladio architectural models. *J. Syst. Softw.* **2014**, *95*, 141–165. [CrossRef]
11. Cortellessa, V.; Marco, A.D.; Eramo, R.; Trubiani, C. Digging into UML models to remove performance antipatterns. In Proceedings of the 32nd International Conference on Software Engineering (ICSE), Cape Town, South Africa, 3 May 2010; Volume 2, pp. 99–108.
12. Navarro, E.; Cuesta, C.E.; Perry, D.E.; González, P. Antipatterns for architectural knowledge management. *Int. J. Inf. Technol. Decis. Mak.* **2013**, *12*, 1133–1160. [CrossRef]
13. Kruchten, P.; Nord, R.L.; Ozkaya, I. Technical Debt: From Metaphor to Theory and Practice. *IEEE Softw.* **2012**, *29*, 18–21. [CrossRef]
14. De Toledo, S.S.; Martini, A.; Sjøberg, D.I.K. Identifying architectural technical debt, principal, and interest in microservices: A multiple-case study. *J. Syst. Softw.* **2021**, *177*, 110968. [CrossRef]
15. Curtis, B.; Sappidi, J.; Subramanyam, J. Measuring the Structural Quality of Business Applications. In Proceedings of the 2011 Agile Conference, Salt Lake City, UT, USA, 7–13 August 2011; pp. 147–150.
16. Zimmermann, T.; Nagappan, N. Predicting Defects Using Network Analysis on Dependency Graphs. In Proceedings of the 33rd International Conference on Software Engineering (ICSE), Leipzig, Germany, 10–18 May 2011; pp. 531–540.
17. Avgeriou, P.; Kruchten, P.; Oskaya, I.; Seaman, C. Managing Technical Debt in Software Engineering. *Dagstuhl Rep.* **2016**, *6*, 110–138.
18. Palomba, F.; Zaidman, A.; Oliveto, R.; De Lucia, A.; Di Penta, M. An Extensive Comparison of Code Smell Detection Tools. *Inf. Softw. Technol.* **2019**, *114*, 1–20.
19. Syed, A.U.H.; Iqbal, J. PAD-A: Performance antipattern detector for AADL. *Int. J. Inf. Technol.* **2021**, *13*, 1383–1391. [CrossRef]
20. Trubiani, C.; Ghabi, A.; Egyed, A. Exploiting traceability uncertainty between software architectural models and performance analysis results. *IFIP Adv. Inf. Commun. Technol.* **2015**, *448*, 104–118.
21. Seaman, C.; Guo, Y. Measuring and Monitoring Technical Debt. In *Advances in Computers*; Elsevier: Amsterdam, The Netherlands, 2011; Volume 82, pp. 25–46.
22. Ernst, N.A.; Bellomo, S.; Oskaya, I.; Nord, R.L.; Gorton, I. Measure it? Manage it? Ignore it? Software practitioners and technical debt. In Proceedings of the 10th International Symposium Empirical Software Engineering and Measurement (ESEM), Torino, Italy, 18–19 September 2014; pp. 1–10.
23. Sweller, J. Cognitive load during problem solving: Effects on learning. *Cogn. Sci.* **1988**, *12*, 257–285. [CrossRef]
24. Yang, C.; Liang, P.; Avgeriou, P. A systematic mapping study on the combination of software architecture and agile development. *J. Syst. Softw.* **2016**, *111*, 157–184. [CrossRef]
25. Kruchten, P. The 4+1 View Model of Architecture. *IEEE Softw.* **1995**, *12*, 42–50. [CrossRef]
26. Hofmeister, C.; Nord, R.; Soni, D. *Applied Software Architecture*; Addison-Wesley: Boston, MA, USA, 2000.
27. Storey, M.A.; Hoda, R.; Milani, A.M.P.; Baldassarre, M.T. Guiding principles for mixed methods research in software engineering. *Empir. Softw. Eng.* **2025**, *30*, 138. [CrossRef]

28. Perez, S.; Diaz-Pace, A.; Martinez, J.F. Identifying Architectural Bad Smells in Software Product Lines. In Proceedings of the 13th International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS), Leuven, Belgium, 6–8 February 2019; pp. 1–8.
29. Structurizr. Available online: https://structurizr.com (accessed on 1 May 2025).
30. Negri-Ribalta, C.; Geraud-Stewart, R.; Sergeeva, A.; Lenzini, G. A systematic literature review on the impact of AI models on the security of code generation. *Front. Big Data* **2024**, *7*. [CrossRef]
31. Shahin, M.; Babar, M.A.; Zhu, L. Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access* **2017**, *5*, 3909–3943. [CrossRef]
32. Aversano, L.; Carpenito, U.; Iammarino, M. An Empirical Study on the Evolution of Design Smells. *Information* **2020**, *11*, 348. [CrossRef]
33. Bavota, G.; Russo, B.; Oliveto, R. Machine learning techniques for software maintainability prediction. *Empir. Softw. Eng.* **2020**, *25*, 3932–3973.
34. Sweller, J.; van Merriënboer, J.J.G.; Paas, F.G.W.C. Cognitive architecture and instructional design. *Educ. Psychol. Rev.* **1998**, *10*, 251–296. [CrossRef]
35. Fagerholm, F.; Guinea, A.S.; Mäenpää, H.; Münch, J. The RIGHT model for Continuous Experimentation. *J. Syst. Softw.* **2017**, *123*, 292–305. [CrossRef]
36. Kazman, R.; Bass, L.; Klein, M.; Clements, P. *The Architecture Tradeoff Analysis Method*; Software Engineering Institute: Pittsburgh, PA, USA, 2005.
37. Curtis, B.; Krasner, H.; Iscoe, N. A field study of the software design process for large systems. *Commun. ACM* **1988**, *31*, 1268–1287. [CrossRef]
38. Schmid, L.; Hey, T.; Armbruster, M.; Corallo, S.; Fuchß, D.; Keim, J.; Liu, H.; Koziolek, A. Software Architecture Meets LLMs: A Systematic Literature Review. *arXiv* **2025**, arXiv:2505.16697. [CrossRef]
39. Zazworka, N.; Shull, F.; Seaman, C.; DeMara, F. Investigating the impact of design debt on software quality. In Proceedings of the Workshop on Managing Technical Debt, Honolulu, HI, USA, 23 May 2011.
40. Oskaya, I. Are DevOps and Automation Our Next Silver Bullet? *IEEE Softw.* **2019**, *36*, 3–95. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.